

Web Mining Project Report: Recommender System on the X-Wines Dataset

Denis Maxheimer, Frederik Röckle, Ursula Sattler, Vincent Schmitt, Fabian Streit

University of Mannheim, Mannheim 68131, Germany

Abstract. We explore classical and graph-based recommender system models on the large-scale X-Wines dataset to recommend wines based on user ratings and wine metadata. Due to the non-existence of dedicated test splits, we developed our own train and test segments for evaluating our recommender systems. By evaluating various models across different cold and hot start scenarios, we show that hybrid approaches outperform standalone models, while graph-based methods face performance and scalability challenges. Our results highlight the effectiveness of feature-rich models. Our code is available under: https://github.com/Frederik-Roeckle/xwines_recom

1 Introduction

Recommender Systems are often used to match users with items. Today, there are large datasets and code examples in the realm of e-commerce or movies available for recommender systems. We wanted to look at a field that has not been explored to such an extent, as the mentioned fields. Whilst in the search for datasets, we came across the X-Wine dataset, which is a large dataset intended to fill the need for a large dataset in the wine domain [2]. Our aim is to apply classical and graph-based recommender system approaches on this dataset, and compare them. Our overarching goal, is to recommend wines, in the best possible way to a user, based on previous ratings and wine attributes. Besides this overarching project goal, our motivation was driven by exploring a dataset only explored by the authors of the dataset, and to experiment, with novel approaches, such as graph-based recommender systems. Furthermore, it was of interest, how different models cope with this particular large-scale dataset.

To model the inherent notion of the ratings we evaluated the task as a 9-class classification models on classification metrics. To further capture the finer nuances in the model performances and to account for the recommender style top-k metrics we also evaluated the models on regression metrics.

This report is structured in the following manner: First we will introduce the dataset, preprocessing steps and the test-set creation in Section 2. Afterwards, the used methods are described in Section 3. Our results and the experimental setting are described and discussed in Section 4&5. In the end a conclusion will be provided in Section 6.

2 Dataset

The X-Wines project provides a large dataset about wines from around the world with explicit 5-star user ratings. It consists at full scale of 21M ratings by 1M anonymized users on 100k wines.[2] Its publicly available under: <https://github.com/rogerioxavier/X-Wines>. The data is provided as two CSV-files. *Reviews.csv* containing all user-wine reviews as a triplet of (UserID, Rating, WineID) and *Wines.csv* containing metadata on the wine instances. The data was collected by the authors in 2022 on the open web and spans across ratings from 2012 to 2021. The dataset offers 17 attributes for the 100k wines such as the acidity, grapes, winery, etc. There is no additional metadata about the users as they are fully anonymized through an ID. Through its large properties the dataset is at similar size like MovieLens 20M with 130k users and 27k items. However, ours has around 10x more users and 3x more items while having a similar amount of rating. This lead to a sparser density regarding the individual ratings per user and wine making the predictions more difficult.

For recommending and evaluating the recommendations, we will use the rating attribute. The rating is provided on a 1-5 star-scale in 0.5 steps, with 1 being the lowest and 5 being the best. The dataset is no traditional shopping dataset as there is no information about price or purchase quantities present. We therefore aim to predict a user's potential interest in certain wines and their rating on such wines.

2.1 General Pre-Processing

General data cleaning and pre-processing was already done by the authors of the X-Wines dataset. There are no duplicate or empty records. The wine-attributes are standardized and appear in a structured format either as strings, numerical values or in form of lists. This reduces the overall effort for pre-processing on our end[2]. Model specific pre-processing was necessary and is explained in the respective sections later on.

2.2 Train and Test Set Creation

The X-Wines dataset itself is not splitted into explicit train or test sets, which are needed to train and to evaluate models appropriately. Therefore, we had to create this ourselves following and paying attention to multiple factors. In recommender systems there is the common distinction between hot and cold, for users and items. Meaning that an entity is hot when the model has already seen it during training time and it is cold when it was not seen at training time. Derived from that, there are four scenarios that can arise:

- Hot/Hot (Known User/Known Wine): Standard case for typical recommendations
- Hot/Cold (Known User/New Wine): A new wine is introduced to already known users

- Cold/Hot (New User/Known Wine): A new user gets some wine recommendation
- Cold/Cold (New User/New Wine): Most challenging case in which a new user is getting a recommendation on a new wine

We therefore created a different test segment for each of the four scenarios. This allows for fine-grained evaluation of the model performance in the different categories later on. The characteristics of the different test-sets are described in table 1. For the split of the train/test sets, we considered three main factors

Characteristics	Hot/Hot	Hot/Cold	Cold/Hot	Cold/Cold
Ratings	2,036,778	35,456	506,800	16,504
Unique users	464,452	27,729	58,969	15,051
Unique wines	83,137	1,321	60,449	1,515
Cold users (%)¹	0.0	0.0	100	100
Cold wines (%)	0.0	100	0.0	100
Mean of user reviews count	4.38	1.27	8.59	1.09

Table 1. Description of test-sets

where we paid close attention to: temporal drift, cold-start problem and distribution similarity. Initially we created a classical 80/20 train/test split which we adjusted according to these three factors. To account for a possible temporal drift, we sorted the ratings chronologically so that ratings from the past were put inside of the training set and the newer rating were reserved for the testing. So that we only predict the future with values from the past and to not introduce any data leakages. The creation of the cold start test segments has been done by mapping the rating-triplets in accordance to their presence of user or wine in the training set. Due to large size differences in the test segments we shortened and sampled additional reviews from the training set which we then removed from there to prevent data leakage. Due to the fact that we had no metadata about the users we would have encountered the issue that for cold user scenarios our models couldn't perform a meaningful prediction as there are no similarities on how to map such a user to the existing ones and their preferences. To account for that, we adapted the cold-user definition towards a low-data cold user. This means that a cold user is now defined as a user that has only two reviews in the training set instead of none. The overall dataset has a class imbalance towards having a high majority for the rating of 3.5 and very low class occurrences for low ratings. We tried to ensure distribution similarity over all training and test sets. However, we could only achieve it to the point where the temporal split was not corrupted by stratified sampling. We evaluated our final training and test sets with the Earth Mover Distance and the Jennings-Shannon divergence showing a close distribution similarity between most of them.

¹ Cold user definition equals to low data users having two reviews in the training set

3 Methods

For the purpose of providing the best recommendation of wines, we built, trained, and evaluated a multitude of different models used for recommender systems. First, we setup a baseline for our models (3.1). Then we looked into classical memory- and model-based approaches (3.2, 3.3). To leverage the metadata available on wines, we implemented a content-based recommender with embeddings and a hybrid recommender system (3.4, 3.5). In the End, we also wanted to look into state-of-the art graph based recommender systems, and implemented two approaches: GraphSAGE and LightGCN (3.6).

Baseline For the purpose of having a comparable baseline for our models we implemented a DummyClassifier. This model predicts the most common class in a classification setting 3.5 and the mean of 3.88 in the regression setting.

3.1 Memory-based

We also wanted to include a classical user-based collaborative filtering approach. However, this approach was not applicable to our dataset, as the standard algorithms did not scale to the size of our dataset, resulting in memory related issues. This is a common issue in classical collaborative filtering approaches [5].

3.2 Model-based

Singular Value Decomposition (SVD) The SVD algorithm is a matrix factorization technique that minimizes the regularized squared error by using Stochastic Gradient Descent [1]. It scales well on large datasets, offering both accuracy and dimensionality reduction. For our use case SVD was chosen due its scalability, since our training set consists of around 16 million rows and the absence of user metadata, which makes user-based prediction difficult. However, SVD does not address the cold-start problem, as it is not possible to make predictions for user or wines that were not seen during training. Therefore we incorporated SVD into a hybrid model that combines collaborative and content-based filtering, allowing us to make better predictions even in cold-start scenarios.

3.3 Content-based

Content based recommender with embeddings Given the rich set of item attributes (wines), we implemented a content-based recommender system to exploit this information. To improve scalability and capture complex, high-dimensional relationships, we used learned embeddings rather than traditional similarity measures [6].

The preprocessing pipeline included lowercasing text, removing extraneous characters, standardizing numerical features, and dropping redundant columns. Embeddings were generated as follows: 'Grapes' and 'Harmonize' were vectorized

using TF-IDF; alcohol content was standardized; and 'RegionName' was hashed to a fixed-dimensional vector to handle high cardinality. These were concatenated into an embedding matrix for similarity computation, limited strictly to training data to prevent test leakage.

Predictions were made by computing the weighted average of a user's previous ratings, with weights determined by the similarity between the target and previously rated wines. Greater similarity increased influence. In cases of insufficient information (e.g., missing features or user history), the global mean rating from the training set was used as a fallback.

For item recommendations, a function retrieved similar wines to the users previous liking, based on the precomputed similarity matrix, enabling stylistic matching based on encoded attributes. To handle cold-start users (lacking rating history), a popularity-based recommender was developed but excluded from final evaluation due to a revised cold-user definition (see Section 2.2).

3.4 Hybrid Recommender System

The hybrid recommender system combines collaborative filtering and content-based filtering. Combining these two can solve the cold start problem, in which users may have no history of rating or items haven't been rated yet. The content based filtering solves this by relying on the wine attributes, making it possible to recommend items without prior ratings. Therefore the hybrid approach merges collaborative filtering that recommends based on what similar users liked with content based filtering, that recommends wine similar to wines a user already likes. This approach is especially useful for our use case, since the rich metadata of the wine makes collaborative filtering very effective, but the complex and subjective taste for wine of users is still captured through the content-based filtering.

Preprocessing For preprocessing a pipeline was implemented for handling diverse feature types efficiently. Vintage was converted to a numerical format and the Date column transformed into a DaysAgo feature. The DaysAgo feature preserved temporal relevance by expressing dates relative to the most recent entry. Categorical features with few values, like Body or Acidity were one-hot encoded. Multi-label features such as Grapes were processed and stored as sparse matrices to avoid memory overload. High cardinality features like User and Wine IDs, as well as Winery and Region names, were encoded using frequency encoding.

Collaborative Filtering features Here we use our UserID and WineID as features, so the model might catch user preferences and historical behavioral trends.

Content-Based features Augmenting training data set with wine metadata will provide more information for models to generalize better and handle cold-start problem for wines.

Hybrid Integration The Collaborative and Content-based features are then merged together. Gradient-boosting models (LightGBM and XGBoost) are trained to predict rating using the combined features. During training the model learns to weigh collaborative features (user and wine interactions) with content based ones (wine attributes).

Temporal feature Date of the rating (DaysAgo) had been used only for hybrid approach and turned out to have high importance for model’s predictions. It makes sense, since there is a trend of significant increase in majority class ratings (4.0) over time (Fig.1(a)), while minority classes almost don’t change. The models capture that importance, however collaborative features and some wine attributes (ABV, Winery, Region) are still crucial for prediction.

3.5 Graph-based Recommender Systems

To enhance our methods for predicting wine ratings and recommending wines, we move from traditional recommender systems to graph-based approaches. By representing (User, Rating, Item) as edges in a bipartite graph, we leverage graph machine learning. Similar users and wines connected through shared edges suggest potential recommendations.

We model users and items as nodes, with ratings as edges. To predict a user’s wine rating, we evaluate edge ratings (RMSE), and for wine recommendations, we assess edge existence (Recall@k). Our method involves constructing a graph from training data, computing node embeddings with a Graph Neural Network (GNN), and using a predictor to estimate ratings based on these embeddings. Training was completed using L4 and A100 GPUs on Google Colab Pro.

GraphSAGE GraphSAGE, a GNN, facilitates inductive representation learning on large graphs.[3] This was crucial for handling new nodes in cold test segments. A strictly transductive GNN couldn’t adapt to unseen nodes. GraphSAGE generates node embeddings by aggregating features from their immediate neighbors. We enhanced node features: users’ features included average rating, count, standard deviation, and rating bias, given the absence of metadata. For wines, we used metadata and computed separate embeddings for *Grapes*, encoding *Type*, *Elaborate*, *Body*, *Acidity*, and *Country*.

Our GNN consists of two SAGEConv layers for embedding encoding and a three-layer MLP as the predictor head. We selected two message-passing layers to capture information from nodes up to two degrees apart. Each user node embedding incorporates data from wines it directly interacted with (1st degree) and from other users who interacted with those wines (2nd degree). This approach, also applicable to wine nodes, helps to capture nuanced similarities between users and wines. Due to cost and time constraints, we couldn’t extensively optimize our model’s hyperparameters, which likely affected performance. We trained our model for ten epochs. Evaluating ranked list metrics revealed our model’s limitations in recommending sets from all wines, as the A100 GPU couldn’t process

the utility function for 100k wine nodes. This mirrors industry scenarios where direct ranking from all items isn't feasible. Instead, a two-stage process is employed: first, a coarse algorithm selects 1k items; second, a detailed model ranks these items. This led to developing our LightGCN model as the initial coarse selector.

LightGCN LightGCN belongs to the class of Graph Convolution Networks (GCN), which are the current SotA for collaborative filtering. It's a light-weight model that only leverages neighborhood aggregation and uses linear propagation between the layers to compute the node embeddings. This makes the model especially light-weight and easy to train.[4] We use a 2-layer LightGCN model from PyG together with Binary Loss to train the model. We choose the Binary Loss over Bayesian Personalized Ranking due to simplification and time limitations. Our key motivation in developing and training the LightGCN model was to develop an industry-like, two-stage recommender system that is able to recommend wines from the full catalog of 100k wines. We followed this approach out of curiosity as we experienced the computational challenges with only using GraphSAGE. As this model setup of GraphSAGE+LightGCN is not comparable to the other models as we evaluate them differently we only developed this pipeline as a proof of concept. We achieved a Recall@20 of 0.0214 on average.

4 Evaluation

In this section we introduce the setup for our evaluation and the results of the models on the different setups.

4.1 Evaluation setup

We trained and evaluated all of our models on the same training and test segments to allow for proper comparison (as defined in 2.2). Some models were additionally finetuned, for that we created a separate validation split from the training segment on a per model basis. In general, we use our models to predict the rating a user would give to a certain wine. Derived from that we have three metrics to evaluate our models. At the core it is a regression task, where we evaluate the nuanced performance with RMSE, in second, we frame the problem as a classification task over the nine distinct categories. We map the continuous ratings by rounding to the next closest class. This allows us to create confusion matrices, visualizing how models perform category-wise. And the last metric, commonly used for recommender systems, are ranked list, where we evaluate the models on Recall@k, Precision@k and NDCG@k. We compare the results of our models against a Baseline model, that is a DummyRegressor always predicting the trainings mean. Furthermore, as we saw that the model performances regarding the RMSE are close together, we used bootstrapped sampling to further solidify the evaluation from a statistical standpoint and to quantify uncertainty. To evaluate the model performances on the ranked list metrics, we followed the

approach presented in the exercise session where the predictions of a test segment were used to calculate the ranked metrics in a closed form. This means that only the items in the test set were available for creating the ranked list. This is an artificial setup compared to the evaluation we created for the LightGCN model. Therefore the metrics between them are not comparable.

4.2 Results

At the core, we have the regression metrics in Table 2. With superior performance across all test segments is the hybrid approach based on SVD. The second and third best performing models are the other hybrid approaches with LightGBM and the XGBoost. The content-based model was always equal or lower compared to the Baseline. Similar performance achieved the GraphSAGE model, while only beating the baseline in the Hot/Hot segment, it performs lower on the other segments.

Models	Full testset	Hot User / Hot Wine	Hot User / Cold Wine	Cold User / Hot Wine	Cold User / Cold Wine
Baseline	0.7016	0.6901	0.6848	0.7434	0.8223
Content-based	0.8556	0.7950	0.6848	1.0739	0.8223
LightGBM	0.6350	0.6179	0.6655	0.6931	0.7659
SVD	0.5891	0.5656	0.6439	0.6658	0.7735
XGBoost	0.6315	0.6130	0.6827	0.6912	0.8264
GraphSAGE	0.7416	0.6505	0.7845	1.0213	0.9675

Table 2. RMSE for Rating Prediction, Bootstrapped mean ($n = 10,000$)

Regarding the ranked list metrics; We achieved similar results with all models in the categories precision@5, recall@5 and ndcg@5. Particular interesting for the recall metric was that we obtained better results for the especially hard cold start scenarios with cold wines ((hot user, cold wine) and (cold user, cold wine)). We visualized the results for them in Figure 1(e). In general, the metrics for precision and recall varied between the 0.6 and 0.9 depending on the testset. For the NDCG metric, across all testsets we achieved values above 0.8. This is particular interesting as our models seems to perform better when it comes to ranking the items. However, the retrieval aspect, monitored by the recall metric is less sophisticated.

The results we obtained from the classification metrics were used to create confusion matrices for enabling a visual error analysis on the models.

5 Discussion

To especially explain the low performance of the GraphSAGE and the Content-based model we also looked into the R^2 metrics for them. The results are concurrent to the RMSE metrics, as both models had consistently a negative R^2 score, meaning that they aren't capable in explaining any variance. To analyze

the performance of the GraphSAGE model, the confusion matrices over the different test segments showed a general bias towards under predicting higher true ratings for cold user segments (see Figure 1(c) and 1(d)). This is reasonable as there is no metadata about the user available.

SVD Best performance is achieved in the hot/hot scenario as expected, since user and wine are known. Performance degrades in cold-start settings, especially when both user and wines are unseen. For R^2 in the hot/hot scenario the value is under 0.30, which means much of the variance is still unexplained. The cold/cold result for R^2 shows poor results, emphasizing the cold-start problem with SVD.

Distribution differences As in general, a model can only predicts reasonable if the target distribution is similar to the learned distribution, we investigated on potential distribution differences that could account for the overall moderate model performances. As described in Section 2.2, the distribution differences on the Rating between the segments where somehow small. However, with the (Hot user, Hot Wine) and the (Cold User, Cold Wine) we have more significant changes in the EMD, that lead to an on average rating shift of 0.14 and 0.18 compared to the training distribution (see Table 3). We researched the

EMD	Hot User / Hot Wine	Hot User / Cold Wine	Cold User / Hot Wine	Cold User / Cold Wine
Train - Test	0.14	0.06	0.04	0.18

Table 3. EMD between trainset and test segments

origins for these shifts and came to the conclusions that the distribution shift for the hot/hot set can be attributed towards a general temporal distribution shift over time as this segment simply accounts for on average of all newer data as seen by the temporal split in Figure 1(b). Together with the shift towards an increased class distribution (3.5, 4.0) as seen in Figure 1(a) we see a clear imbalance between train and testset distribution, that at least partly accounts for the performance. The distribution shift for the cold/cold segment arise probably through the random sampling of new users and items from the training set and is not averaged out due to the small sample size of $n = 500$.

Graph approach As the graph models were extreme compute intense, we couldn't invest resources in hyperparameter tuning or model architecture design. Also the possibilities for encoding the meta information into the graph structure could be more improved and evaluated. We see potential for adding additional node types like country, region or winery to create more global neighborhoods that would help in increasing the recall performance on larger scales. Another design choice that we would challenge are the 2 message-passing layers that we chose. Adding another layer would also capture wine-user-wine relationships that would potentially help aligning users embeddings closer together based on similar wine preferences. For training the LightGCN model a straightforward improvement potential would be seen in using BPR instead of Binary Loss as we

then train the model on a per user basis which would help capturing the lower represented class labels as they aren't overwritten with global majority classes.

Ranked List Evaluation Setup The ranked-list evaluation as we did it, is easy and scales well. However, it artificially narrows the search space for the items and introduces biases as the wines in the test segment are with guarantee rated by the user. This approach alters the real setting as how it would be seen in industry where the ranked list metrics would be created across all items in the catalog. We did this for our LightGCN model in Section 3.6. However, it is more complex and limited by the compute power.

Another aspect was that we don't get any results for the cold-cold set when choosing $k \geq 10$. The origin for this is, that the cold-cold test set has only 23 users with 5 or more ratings out of 15k users in the segment. Therefore, to have a common ground, we could only compare all the segments with $k = 5$. Further exploration of the test segments lead to the discover that on average each user only rates one wine in the (hot user, cold wine) and (cold user, cold wine) segments as compared to 4 and 8 wines in the other segments. The models in these scenarios obtain a recall of 1.0 automatically if only one out of their 5 retrieved wines is correct. For further improvement, it would be necessary to stratify the test segments not only on Ratings but also on amount of users and wines to enable better and more compareable evaluation.

Metadata-feature ablation study As the wine metadata provides a rich context of information about each wine, not all of them might be equal qualified to increase the model performance. It would therefore be insightful to conduct an in-depth ablation study on how different subsets would influence the models performance. This would lead to improved feature sets for training our models as well as in-depth information about certain limitations of the current models which would be of help to improve the overall generalization capabilities. However, due to time restrictions and the vast complexity of the wine metadata features, we weren't able to conduct it as we would have liked to do it.

6 Conclusion

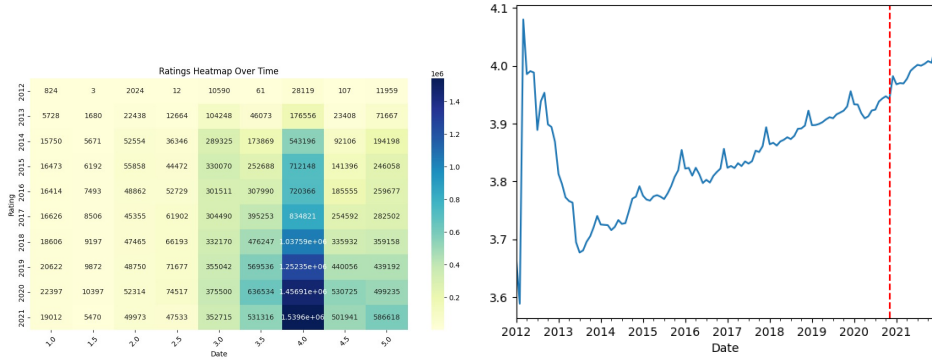
In this project we implemented and evaluated multiple recommender system models for predicting wine for users under hot and cold start scenarios. We found that the hybrid approaches, that combine collaborative filtering with content-based filtering outperformed the individual models. This confirms the hypothesis that combining both models can help mitigate the weaknesses of each, especially in our use-case in which the wine metadata is rich but the user history is weak. The SVD model achieved the best performance, especially in hot-start scenarios. The performance dropped in cold-start scenarios, confirming the limitation of SVD when user or wine history is missing. The content-based and GraphSAGE models mostly failed to outperform the baseline, with particularly poor performance in cold-start scenarios. GraphSAGE was a theoretically a promising approach but probably underperformed due to limited overall training and

potential issues with the graph structure. Even a coarse hyperparameter optimization could lead to more promising results. The confusion matrix revealed a bias toward under-predicting higher ratings in cold user scenarios. Further, our evaluation revealed distribution shifts between training and test sets in the hot/hot and cold/cold segments. In general we can say that all models had a significant performance drop in cold-start scenarios. Another takeaway from our evaluation is the limitation of the ranked list setup, that is efficient and easy to scale, but artificially narrows the candidate pool by only evaluating on wines already present in the test set, which introduces a bias. This does not reflect real-world setup where the full catalog is considered. For future research, it would be particularly interesting to conduct an in-depth feature ablation study to discover the impact of each attribute on the overall model performances and also detect additional weaknesses in our models. Overall, this project enabled us to get a broad deep dive into the world of classical recommender systems and graph-based alternatives. By learning and improving around the central challenges in these domains we improved our theoretical knowledge of the field and further fostered our engineering capabilities.

References

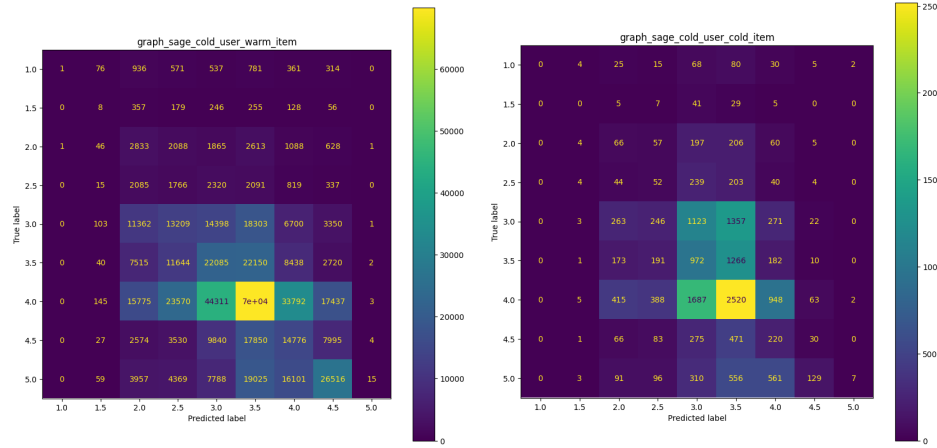
1. Matrix factorization-based algorithms — surprise 1 documentation, https://surprise.readthedocs.io/en/stable/matrix_factorization.html
2. De Azambuja, R.X., Morais, A.J., Filipe, V.: X-Wines: A Wine Dataset for Recommender Systems and Machine Learning. *Big Data and Cognitive Computing* **7**(1), 20 (Jan 2023). <https://doi.org/10.3390/bdcc7010020>, <https://www.mdpi.com/2504-2289/7/1/20>
3. Hamilton, W.L., Ying, R., Leskovec, J.: Inductive Representation Learning on Large Graphs (Sep 2018). <https://doi.org/10.48550/arXiv.1706.02216>, <http://arxiv.org/abs/1706.02216>, arXiv:1706.02216 [cs]
4. He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., Wang, M.: LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation (Jul 2020). <https://doi.org/10.48550/arXiv.2002.02126>, <http://arxiv.org/abs/2002.02126>, arXiv:2002.02126 [cs]
5. Sarwar, B.M.: Challenges of User-based Collaborative Filtering Algorithms (Feb 2001), <https://ra.ethz.ch/CDstore/www10/papers/519/node9.html>
6. Zhao, X., Wang, M., Zhao, X., Li, J., Zhou, S., Yin, D., Li, Q., Tang, J., Guo, R.: Embedding in Recommender Systems: A Survey (Dec 2023). <https://doi.org/10.48550/arXiv.2310.18608>, <http://arxiv.org/abs/2310.18608>, arXiv:2310.18608 [cs]

A Appendix



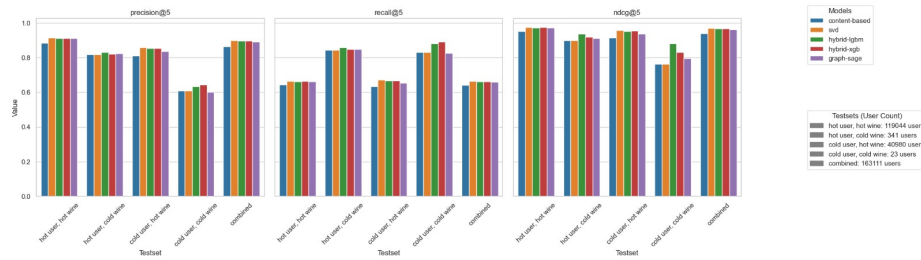
(a) Ratings over time

(b) Temporal drift for Average Ratings



(c) CM GraphSAGE for cold/warm

(d) CM GraphSAGE for cold/cold



(e) Ranked List metrics

Fig. 1. (a) and (b) indicates clear temporal drift of increasing on average rating over time and move towards majority class. (c) and (d) show confusion matrix for GraphSAGE indicating bias to underestimate ratings. (d) ranked list metrics